

Design an In-Memory File System

Google · Amazon · Microsoft · Uber – LeetCode 588 గా ప్రసిద్ధం, కానీ interview lo అది మొదటి పది నిమిషాలు మాత్రమే.

మూడు విరుపులు. మొదటిది `ls` ని **4,620 ms** చేస్తుంది – ఎప్పుడూ 20 పేర్లు ఇచ్చినా. రెండోది `/etc/passwd` ని **బయటికి** ఇచ్చేస్తుంది. మూడోది **stack** ని పేల్చేస్తుంది.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి సమయం, ఆ `RangeError` తో సహా – నిజంగా `node` lo run చేసినదే.

ఈ **PROBLEM** ఒక ఉచ్చు – మరియు అది ఎందుకే ముందే తెలుసుకోండి

"File system design చెయ్యి" అంటే చాలా మంది వెంటనే `Map<string, string>` రాస్తారు – path నుంచి content కి. అది పనిచేస్తుంది, tests pass అవుతాయి, మరియు 10 నిమిషాల్లో పూర్తవుతుంది.

ఆపై interviewer అడుగుతాడు: "ఒక directory పేరు మార్పు", "ఖాళీ directory ఎలా?", "ఇది నెమ్మదిగా ఉందా?" – మరియు ఆ `Map` ఒక్కొక్క ప్రశ్నకీ విరిగిపోతుంది.

ఈ doc lo ఆ మూడు ప్రశ్నలనీ ముందే అడిగి, కొలిచి, సరిచేస్తాం.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఎక్కడ లోతు ఉంది
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: path ఒక తాళంచెవి కాదు

3. Step – పూర్తి path → content అనే Map
4. మొదటి విరుపు – 20 పేర్లు ఇవ్వడానికి 4,620 ms
5. Step – Directory ఒక నిజమైన వస్తువు

Part 3 – రెండో విరుపు: `..` అనే రెండు అక్షరాలు

6. Step – `path.split('/')` సరిపోతుందా?
7. రెండో విరుపు – ఉన్న file దొరకలేదు · లేని file దొరికింది
8. Step – సాధారణీకరించడం, ఆపై లోపలేనా అని తనిఖీ

Part 4 – మూడో విరుపు: చెట్టు చెట్టు కాదు

9. Step – symlink: ఒక పేరు, మరొక చోటు
10. మూడో విరుపు – `RangeError: Maximum call stack size exceeded`
11. Step – "ఎప్పుడైనా చూశామా" కాదు, "ఇప్పుడు ఈ దారిలో ఉందా"

Part 5 – పూర్తి system

12. Step – ఒక subdirectory ని root గా చూపించడం
13. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

Part 6 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
15. నోటీతో చెప్పాల్సిన English script
16. Follow-ups – permissions, size cache, persistence
17. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఎక్కడ లోతు ఉంది

"Design an in-memory file system supporting mkdir, ls, addContentToFile and readContentFromFile."

ఇది LeetCode 588. మరియు ఆ నాలుగు operations ని ఒక Map తో 15 పంక్తుల్లో రాయొచ్చు. **అదే ఉచ్చు.**

ఒక సులభమైన ప్రశ్న • మూడు లోతైన తలుపులు

"mkdir, ls, read, write" – ఆపై interviewer తలుపు తడతాడు

1 • "ఇది నెమ్మదిగా ఉందా?"

Map lo ls అంటే
ప్రతి key నీ చూడటం
జవాబు 20 పేర్లే అయినా

→ §4 • 4,620 ms

2 • ".. ఎలా పనిచేస్తుంది?"

రెండు అక్షరాలు –
కానీ వీటి వల్లే ప్రపంచంలో
అత్యధిక web దాడులు

→ §7 • /etc/passwd

3 • "Symlinks చేర్చు"

ఇక్కడ చెట్టు ఆగిపోతుంది –
ఇది ఇప్పుడు ఒక **graph**
మరియు graph lo వలయాలు ఉంటాయి

→ §10 • stack పీలుతుంది

ఈ problem యొక్క నిజమైన ఆకారం

మొదటి పరిష్కారం **తప్పు కాదు** – అది ఒక Map, మరియు అది అడిగిన నాలుగు పనులూ చేస్తుంది.

కానీ అది **ఒక తప్పు ఆకారాన్ని** ఎంచుకుంది: path ని ఒక విడదీయలేని తాళించెవిగా చూసింది.

నిజానికి path అనేది **ఒక దారి** – అంటే దానిలో నిర్మాణం ఉంది, మరియు ఆ నిర్మాణమే అసలు డేటా.

ఒకసారి ఆ ఆకారం తప్పితే, మిగతా మూడూ దానంతట అవే వస్తాయి.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
ఖాళీ directory ఉండగలదా?	అవును అంటే Map ఇక్కడే చనిపోతుంది – content లేని entry ఎలా?
mv / rename కావాలా?	అవును అంటే §4 – ఇదే అతి పెద్ద తేడా

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
. మరియు .. ఉంటాయా?	అవును అంటే §8 – మరియు ఇది భద్రతా ప్రశ్న కూడా
Symlinks / hard links?	అవును అంటే ఇది చెట్టు కాదు, graph (§9)
ఎన్ని files? ls ఎంత తరచుగా?	§4 lo ఇదే కొలత
Permissions, owners?	సాధారణంగా వదిలేయమంటారు – అడగండి (§16)
Thread-safe కావాలా?	అడగండి; సాధారణంగా single-threaded

మొదటి ప్రశ్న ఒక్కటే మీ మొత్తం DESIGN ని నిర్ణయిస్తుంది

"ఒక ఖాళీ directory ఉండగలదా?"

జవాబు అవును. `mkdir /home/ravi/empty` – అందులో ఏమీ లేదు, కానీ అది ఉంది.

ఇప్పుడు `Map<path, content> lo` దాన్ని ఎలా నిల్వ చేస్తారు? Content లేదు.

`Map.set('/home/ravi/empty', '')` అంటే – అది ఒక ఖాళీ file నా, ఒక ఖాళీ directory నా? ఆ Map కి తేడా తెలియదు.

ఈ ఒక్కప్రశ్న Map ని అసాధ్యం చేస్తుంది. మరియు చాలా మంది దీన్ని §4 వరకు గమనించరు.

Part 2 – మొదటి విరుపు: path ఒక తాళంచెవి కాదు

3. Step – పూర్తి path → content అనే Map

అతి సహజమైన మొదటి version:

```
class FlatFS {
  constructor() { this.files = new Map(); }    // "/a/b/notes.txt" → "hello"

  write(path, content) { this.files.set(path, content); }
  read(path) { return this.files.get(path); }

  ls(dir) {
    const prefix = dir.endsWith('/') ? dir : dir + '/';
    const out = new Set();
    for (const p of this.files.keys())        // ప్రతి file నీ చూస్తుంది
      if (p.startsWith(prefix))
        out.add(p.slice(prefix.length).split('/')[0]);
    return [...out].sort();
  }
}
```

చూడటానికి బాగానే ఉంది, మరియు పనిచేస్తుంది:

```
ls('/home')      : [ 'asha', 'ravi' ]
ls('/home/ravi') : [ 'notes.txt', 'photos', 'todo.md' ]
read notes.txt   : hello
```

ls('/home') 'ravi' మరియు 'asha' ఇచ్చింది – /home/ravi/photos/goa.jpg అనే మూడు-అంచెల path నుంచి కూడా సరైన ఒక్క పేరే తీసింది. తెలివైనది.

మరి సమస్య ఏమిటి?

4. మొదటి విరుపు – 20 పేర్లు ఇవ్వడానికి 4,620 ms

సమస్య 1 – ఖాళీ directory అనేది లేనే లేదు

```
mkdir('/home/ravi/empty') తర్వాత ls('/home/ravi'):
[ 'notes.txt', 'photos', 'todo.md' ] ← empty ఎక్కడ?
```

`mkdir` అనే method **రాయనే లేదు** – ఎందుకంటే రాయడానికి ఏమీ లేదు. Map lo files మాత్రమే ఉంటాయి; directories అనేవి పేర్లలో దాగిన **భ్రమ** మాత్రమే. `photos` కనిపిస్తోంది ఎందుకంటే అందులో ఒక file ఉంది. `empty` lo ఏమీ లేదు, కాబట్టి **అది ఉనికిలోనే లేదు**.

సమస్య 2 – ఒకే చోటికి పలు పేర్లు

ఒకే directory, మూడు రకాలుగా రాస్తే:

```
ls('/home/ravi')      → ["notes.txt", "photos", "todo.md"]
ls('/home/ravi/')     → ["notes.txt", "photos", "todo.md"]
ls('/home//ravi')     → []
```

`/home//ravi` – ఇది సరిగ్గా అదే directory. కానీ string మ్యాచ్ కాదు, కాబట్టి **ఖాళీ**.

సమస్య 3 – మరియు ఇదే అసలు సమస్య: కొలిచి చూద్దాం

`/data/d0` lo **ఎప్పుడూ సరిగ్గా 20 files** ఉంచుదాం. మిగతా files వేరే చేటు. ఆపై `ls('/data/d0')` ని 2,000 సార్లు పిలుద్దాం:

మొత్తం files	2,000 ls calls	rename /other
1000	16 ms	0 ms, 980 keys
50000	516 ms	10 ms, 49980 keys
500000	4786 ms	212 ms, 499980 keys

జవాబు మారలేదు • ఖర్చు 300× పెరిగింది

`ls('/data/d0')` ఎప్పుడూ అదే 20 పేర్లు ఇస్తుంది – మూడు సందర్భాల్లోనూ

1,000 files	16 ms
50,000 files	516 ms
500,000 files	4,786 ms – అదే 20 పేర్ల కోసం

ఇది ఒక **నియమాన్ని ఉల్లంఘిస్తోంది**: ఒక operation ఖర్చు దాని **జవాబు** పరిమాణంతో పెరగాలి, మొత్తం డేటా పరిమాణంతో కాదు. ఇక్కడ జవాబు 20 – కానీ ఖర్చు 5,00,000. మరియు ఒక directory పేరు మార్చడానికి **4,99,980 keys** తిరగరావాలి.

మొదటి విరుపు – PATH ని ఒక తాళించెవిగా చూశాం

జవాబు ఎప్పుడూ 20 పేర్లు. ఖర్చు 16 ms → 4,786 ms. 500× ఎక్కువ files, 300× నెమ్మది – మరియు ఆ files వేరే **directory** lo ఉన్నాయి.

మరియు rename: ఒక directory పేరు మార్చడానికి **4,99,980 keys** తొలగించి మళ్ళీ చేర్చాలి. నిజమైన file system lo అది **ఒక్క pointer** మార్పు.

మౌలిక తప్పు: `/home/ravi/notes.txt` అనేది ఒక తాళించెవి కాదు – అది **ఒక దారి**. అందులో `home`, `ravi`, `notes.txt` అనే మూడు వేర్వేరు విషయాలు ఉన్నాయి, మరియు వాటి మధ్య **సంబంధాలు** ఉన్నాయి. String lo వాటిని కలిపేస్తే – ఆ సంబంధాలని ప్రతిసారీ తిరిగి కనుక్కోవాల్సి వస్తుంది.

5. Step – Directory ఒక నిజమైన వస్తువు

ఆ నిర్మాణాన్ని నిల్వ చేద్దాం, ప్రతిసారీ తిరిగి లెక్కించకుండా:

```
class FsNode {
  constructor(name, parent = null) { this.name = name; this.parent = parent; }
  get path() {
    if (!this.parent) return '/';
    const up = this.parent.path;
    return up === '/' ? '/' + this.name : up + '/' + this.name;
  }
}

class FileNode extends FsNode {
  constructor(name, parent, content = '') {
    super(name, parent); this.content = content;
  }
  get isDir() { return false; }
  get size() { return this.content.length; }
}

class DirNode extends FsNode {
  constructor(name, parent = null) {
    super(name, parent); this.children = new Map();
  }
  get isDir() { return true; }
  child(n) { return this.children.get(n) ?? null; }
  put(node) { this.children.set(node.name, node); return node; }
}
```

మూడు వివరాలు:

- **children ఒక Map** – పేరు నుంచి node కి. అంటే `ls` అంటే `[...children.keys()]` – **O(children)**, మొత్తం files తో సంబంధం లేదు.
- **parent ఒక pointer** – అందుకే `path` ని లెక్కించగలం. Path ఇప్పుడు నిల్వ చేసిన డేటా కాదు, **ఒక ఉత్పన్న విలువ**.
- **DirNode మరియు FileNode వేర్వేరు** – ఖాళీ directory ఇప్పుడు పూర్తిగా సాధారణమైనది: ఖాళీ `children` Map.

నడక ఒక్క చోట:

```
#walk(parts, { create = false } = {}) {
  let cur = this.root;
  for (const part of parts) {
    let next = cur.child(part);
    if (!next) {
      if (!create) return null;
      next = cur.put(new DirNode(part, cur)); // లేని parents ని సృష్టించడం
    }
    if (!next.isDir) return null;
    cur = next;
  }
}
```

```

    return cur;
}

```

మరియు rename – ఒక్క pointer మార్పు:

```

rename(from, to) {
  const fp = from.split('/').filter(Boolean), fname = fp.pop();
  const tp = to.split('/').filter(Boolean), tname = tp.pop();
  const fromDir = this.#walk(fp), node = fromDir && fromDir.child(fname);
  if (!node) return { ok: false, reason: `NOT_FOUND: ${from}` };
  const toDir = this.#walk(tp, { create: true });
  fromDir.children.delete(fname);
  node.name = tname; node.parent = toDir; // ← ఇదే మొత్తం పని
  toDir.put(node);
  return { ok: true, moved: node.path };
}

```

అందులోని subtree – వేలాది files – కదలలేదు. వాటి path దానంతట అదే కొత్తది అవుతుంది, ఎందుకంటే అది parent గొలుసు నుంచి లెక్కించబడుతుంది.

అదే కొలత మళ్ళీ

మొత్తం files	2,000 ls calls	rename /other
	flat → tree	flat → tree
1000	14 ms → 1 ms	1 ms → 0 ms
50000	492 ms → 1 ms	12 ms → 0 ms
500000	4620 ms → 1 ms	289 ms → 0 ms

Tree వరుస మూడు సందర్భాల్లోనూ 1 ms. అది "చాలా వేగం" అని కాదు – అది **files సంఖ్యతో సంబంధమే లేదు** అని. అదే అసలు విషయం.

ఈ సంఖ్యలు ఎంత స్థిరమైనవి? పలుమార్లు నడిపితే flat వరుస 14-16 ms, 489-516 ms, **4,360-5,100 ms**; tree వరుస ఎప్పుడూ 1-2 ms. ఆ 5,00,000 వరుసలో ~15% ఊగిసలాట ఉంది (GC, memory pressure), కాబట్టి "4,620" అనేది ఒక ఖచ్చితమైన విలువ కాదు.

కానీ అది ముఖ్యం కాదు – ఇక్కడ చూడాల్సినది ఒక సంఖ్య కాదు, **ఒక ఆకారం**: ఒక వరుస డేటాతో పాటు పెరుగుతోంది, రెండేది పెరగట్లేదు.

```

ls('/home/ravi') : [ 'empty', 'notes.txt', 'photos', 'todo.md' ] ← empty ఇప్పుడు
కనిపిస్తోంది ✓

rename /home/ravi/photos → /home/asha/album:
  { ok: true, moved: '/home/asha/album' }
ls('/home/ravi') : [ 'empty', 'notes.txt', 'todo.md' ]
ls('/home/asha') : [ 'album', 'resume.pdf' ]
read goa.jpg     : <binary>

```

Interview lo ఈ వాక్యం చెప్పండి: "An operation's cost should scale with the size of its answer, not the size of the database." – **ls** ఇరవై పేర్లు ఇస్తుంటే, అది ఇరవై పనులే చేయాలి. ఈ ఒక్క వాక్యం Deep Dive 08 (HashMap), 10 (Meeting Scheduler §8), 11 (Food Delivery §8) – మూడింటిలోనూ వచ్చింది.

Part 3 – రెండో విరుపు: `..` అనే రెండు అక్షరాలు

6. Step – `path.split('/')` సరిపోతుందా?

చెట్టు వచ్చింది. Path ని ముక్కలుగా చేసి నడుస్తున్నాం:

```
const parts = path.split('/').filter(Boolean);
```

`filter(Boolean)` ఖాళీ ముక్కలని తీసేస్తుంది, కాబట్టి `//` మరియు చివరి `/` దానంతట అవే పరిష్కారమవుతాయి. బాగుంది.

మరి `.` మరియు `..` ఏమిటి?

7. రెండో విరుపు – ఉన్న file దొరకలేదు • లేని file దొరికింది

మొదటి సగం – ఉన్నది దొరకట్లేదు

<code>/home/ravi/notes.txt</code>	→ hello	సాధారణం
<code>/home//ravi//notes.txt</code>	→ hello	రెండు slashes
<code>/home/ravi/notes.txt/</code>	→ hello	చివర slash
<code>/home./ravi/notes.txt</code>	→ null	<code>.</code> (ఇదే directory)
<code>/home/ravi../ravi/notes.txt</code>	→ null	<code>..</code> తర్వాత తిరిగి
<code>/home/asha../ravi/notes.txt</code>	→ null	పక్క directory కి

చివరి మూడూ `null`. File ఉంది, కానీ ఆ దారిలో దొరకట్లేదు – ఎందుకంటే code `.` మరియు `..` అనే పేర్లున్న children కోసం వెతుకుతోంది.

రెండో సగం – మరియు ఇదే ప్రమాదకరమైనది

సరే, `..` ని అమలు చేద్దాం. సహజమైన రూపం:

```
function naiveResolve(base, path) {
  const parts = (path.startsWith('/') ? path : base + '/' + path).split('/');
  const out = [];
  for (const p of parts) {
    if (p === '' || p === '.') continue;
    if (p === '..') out.pop(); // ← ఒక అడుగు వెనక్కి
    else out.push(p);
  }
}
```

```
return '/' + out.join('/');
}
```

సాధారణ సందర్భాలు **అన్నీ సరిగ్గా** పనిచేస్తాయి:

సాధారణ సందర్భాలు – సరిగ్గానే ఉన్నాయి:

/home/./ravi/notes.txt	→ /home/ravi/notes.txt
/home/ravi/./ravi/notes.txt	→ /home/ravi/notes.txt
/home//ravi//notes.txt	→ /home/ravi/notes.txt
/home/asha/./ravi/notes.txt	→ /home/ravi/notes.txt

ఇప్పుడు ఒక server `/var/www` లోపలి files ని మాత్రమే ఇవ్వాలనుకుందాం:

ఒక server `/var/www` లోపలిది మాత్రమే ఇవ్వాలి. User అడిగేది:

index.html	→ /var/www/index.html	లోపల ✓
img/logo.png	→ /var/www/img/logo.png	లోపల ✓
../../etc/passwd	→ /etc/passwd	బయట ✗
a/b/../../../../../etc/shadow	→ /etc/shadow	బయట ✗

రెండో విరుపు – ఇది ఒక BUG కాదు, ఇది ఒక దాడి

User `../../etc/passwd` అని అడిగాడు, మరియు `naiveResolve` విధేయంగా `/etc/passwd` ఇచ్చింది. Server ఆ file ని చదివి పంపుతుంది.

దీనికి ఒక పేరు ఉంది: **path traversal** (CWE-22). ఇది ప్రపంచంలో అత్యంత సాధారణమైన web దుర్బలత్వాలలో ఒకటి, మరియు అది **సరిగ్గా ఈ code నుంచే** వస్తుంది.

గమనించండి – naiveResolve తప్పు కాదు. అది `..` ని సరిగ్గానే నిర్వహించింది. తప్పు ఏమిటంటే – **అది తన పని పూర్తయిందని అనుకుంది.** సాధారణీకరణ ఒక అడుగు; **పరిధిని తనిఖీ చేయడం** వేరే అడుగు, మరియు అది తప్పనిసరి.

8. Step – సాధారణీకరించడం, అపై లోపలేనా అని తనిఖీ

రెండు వేర్వేరు పనులని రెండు వేర్వేరు functions గా విడదీద్దాం:

```
function normalize(base, path) {
  const raw = path.startsWith('/') ? path : base + '/' + path;
  const out = [];
  for (const p of raw.split('/')) {
    if (p === '' || p === '.') continue;
    if (p === '..') { out.pop(); continue; } // root దాటితే ఖాళీనే ఉంటుంది
    out.push(p);
  }
  return '/' + out.join('/');
}

function resolveInside(jail, path) {
```

```
const full = normalize(jail, path);
const fence = jail.endsWith('/') ? jail : jail + '/';
if (full !== jail && !full.startsWith(fence))
  return { ok: false, reason: `OUTSIDE_ROOT: ${full}` };
return { ok: true, path: full };
}
```

```
/var/www jail:
index.html          { ok: true, path: '/var/www/index.html' }
img/logo.png       { ok: true, path: '/var/www/img/logo.png' }
./img/./logo.png    { ok: true, path: '/var/www/img/logo.png' }
../../etc/passwd     { ok: false, reason: 'OUTSIDE_ROOT: /etc/passwd' }
a/b/../.././../etc/shadow { ok: false, reason: 'OUTSIDE_ROOT: /etc/shadow' }
```

మూడు సూక్ష్మమైన వివరాలు – ప్రతిదీ ముఖ్యమైనది

1 • తనిఖీ తర్వాత, ముందు కాదు. చాలా మంది raw string `lo ..` ఉందా అని చూస్తారు –

"`path.includes('..')` అయితే తిరస్కరించు". అది **విఫలమవుతుంది**: URL encoding (`%2e%2e`), `....//`, Unicode రూపాంతరాలు – అన్నీ దాన్ని దాటిపోతాయి. **సాధారణీకరించిన తర్వాత మాత్రమే** తనిఖీ నమ్మదగినది.

2 • ఆ చివరి / ఒక bug కి తేడా:

```
startsWith("/var/www") → true ← "/var/wwwevil" లోపలిదని చెప్తుంది ✗
startsWith("/var/www/") → false ← సరైనది ✓
```

`/var/wwwevil/secrets.txt` అనేది `/var/www` తో మొదలవుతుంది – కానీ అది **పూర్తిగా వేరే directory**.

3 • Root దాటి పైకి వెళ్ళలేము. `out.pop()` ఖాళీ array మీద ఏమీ చేయదు, కాబట్టి `/../../..` అనేది `/` గానే ఉంటుంది. ఇది ప్రమాదవశాత్తు సరైనది – కానీ **దాన్ని ఒక వ్యాఖ్యగా రాయండి**, లేకపోతే ఎవరో దాన్ని "సరిచేస్తారు".

ఇది ఒక సార్వత్రిక నమూనా

సాధారణీకరించు → ఆపై ధ్రువీకరించు. ఈ క్రమం తప్పితే ప్రతిసారీ ఒక దుర్బలత్వం వస్తుంది:

- **URLs** – redirect కి ముందు host ని సాధారణీకరించండి, లేకపోతే open redirect
- **Emails** – `User@Gmail.com` vs `user@gmail.com` – ఒకే ఖాతా
- **SQL / shell** – escape చేసి, ఆపై పంపండి – ముందు తనిఖీ చేసి కాదు
- **Unicode** – ఒకే అక్షరానికి పలు encodings

నియమం: రెండు రూపాలున్న ఏ ఇన్పుట్నైనా, ముందు ఒకే రూపానికి తెచ్చి, ఆ రూపం మీద నిర్ణయం తీసుకోండి.

Part 4 – మూడో విరుపు: చెట్టు చెట్టు కాదు

9. Step – symlink: ఒక పేరు, మరొక చోటు

Interviewer అంటాడు: "Symlinks చేర్చండి."

ఒక link అంటే – ఒక పేరు, ఇంకో node ని చూపిస్తుంది:

```
class LinkNode {
  constructor(name, parent, target) {
    Object.assign(this, { name, parent, target });
  }
  get isDir() { return false; }
  get isLink() { return true; }
  get size() { return 0; }
}
```

మరియు అన్ని files ని వెతికే ఒక function, links ని అనుసరిస్తూ:

```
function findAll(node, path = '') {
  if (!node.isDir) {
    if (node.isLink) return findAll(node.target, path); // link ని అనుసరించడం
    return [path];
  }
  const out = [];
  for (const [name, child] of node.children)
    out.push(...findAll(child, path + '/' + name));
  return out;
}
```

Ravi తన folder lo asha folder కి ఒక link పెట్టాడు – `/home/ravi/shared` → `/home/asha` :

అన్ని files (links ని అనుసరిస్తూ):

- `/home/ravi/notes.txt`
- `/home/ravi/work/report.pdf`
- `/home/ravi/shared/resume.pdf`
- `/home/asha/resume.pdf`

పనిచేస్తోంది. `resume.pdf` రెండు దారుల్లో కనిపిస్తోంది – అదే link యొక్క ఉద్దేశం.

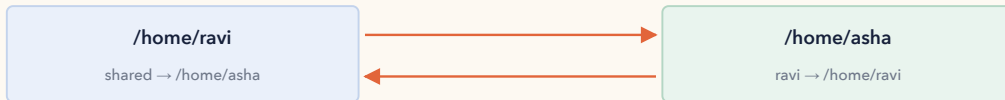
10. మూడో విరుపు – RangeError: Maximum call stack size exceeded

ఇప్పుడు asha కూడా అదే చేసింది – `/home/asha/ravi → /home/ravi`:

```
ఇప్పుడు asha కూడా ravi కి link పెట్టింది:
/home/asha/ravi → /home/ravi
x RangeError: Maximum call stack size exceeded
```

ఇద్దరూ ఒకరినొకరు చూపించుకున్నారు

ఇద్దరూ చేసింది సహజమే – కలిసి ఒక అనంత వలయం



చెట్టు అనే భావనే ఇక్కడ పోయింది

`/home/ravi/shared/ravi/shared/ravi/shared/...` – ఇది ఒక చెట్టుబాటయ్యే path, మరియు అనంతం.

Symlinks చేర్చిన క్షణం, file system చెట్టు కాదు – ఒక graph. Graph lo వలయాలు ఉంటాయి.

మరియు వలయం ఉన్న graph ని చెట్టులా నడిస్తే – stack పేలుతుంది.

మొదటి పరిష్కార ప్రయత్నం – మరియు అది ఎందుకు తప్పు

సహజమైన జవాబు: "చూసిన nodes ని ఒక Set lo పెట్టి, మళ్ళీ చూడకుండా ఉంటే సరి."

```
function findEverSeen(node, path = '', seen = new Set()) {
  if (seen.has(node)) return [];
  seen.add(node);
  ...
}
```

Stack పేలలేదు. కానీ ఫలితం చూడండి:

```
A · "ఎప్పుడైనా చూశామా" set:
/home/ravi/notes.txt
/home/ravi/work/report.pdf
/home/ravi/shared/resume.pdf
```

`/home/asha/resume.pdf` మాయమైంది.

అది ఒక link కాదు – అది asha యొక్క నిజమైన file, ఆమె నిజమైన folder lo. కానీ traversal అక్కడికి చేరేసరికి, asha అనే directory ఇప్పటికే "చూసినది" – ఎందుకంటే `/home/ravi/shared` ద్వారా దాన్ని ముందే తాకాం.

Crash కంటే ఇది ప్రమాదకరం. Crash కనిపిస్తుంది. ఇది కనిపించదు – backup script నిశ్శబ్దంగా కొన్ని files ని వదిలేస్తుంది, మరియు మీకు అది అవసరమైన రోజే తెలుస్తుంది.

11. Step – "ఎప్పుడైనా చూశామా" కాదు, "ఇప్పుడు ఈ దారిలో ఉందా"

సరైన ప్రశ్న ఇది కాదు: "ఈ directory ని ఎప్పుడైనా చూశామా?" సరైన ప్రశ్న ఇది: "ఈ directory ఇప్పుడు నేను నిలబడ్డ దారిలోనే ఉందా?"

ఎందుకంటే వలయం అంటే – మీరు వచ్చిన దారిలోకే తిరిగి వెళ్ళడం. వేరే కొమ్మ ద్వారా అదే directory ని మళ్ళీ చూడటం వలయం కాదు; అది ఒక చెల్లుబాటయ్యే రెండే దారి.

```
function findOnPath(node, path = '', onPath = new Set(), loops = []) {
  if (!node.isDir) {
    if (!node.isLink) return [path];
    if (onPath.has(node.target)) { loops.push(path); return []; }
    return findOnPath(node.target, path, onPath, loops);
  }
  onPath.add(node);
  const out = [];
  for (const [n, c] of node.children)
    out.push(...findOnPath(c, path + '/' + n, onPath, loops));
  onPath.delete(node); // ← తిరిగి వచ్చాక తీసేయడం
  return out;
}
```

ఆ `onPath.delete(node)` ఒక్క పంక్తి మొత్తం తేడా. `add` చేసి, లోపలికి వెళ్ళి, బయటికి వచ్చాక తీసేయడం – అంటే set lo ఎప్పుడూ ప్రస్తుత దారి మాత్రమే ఉంటుంది.

```
B • "ఇప్పుడు ఈ దారిలో ఉందా" set:
/home/ravi/notes.txt
/home/ravi/work/report.pdf
/home/ravi/shared/resume.pdf
/home/asha/resume.pdf
/home/asha/ravi/notes.txt
/home/asha/ravi/work/report.pdf
వలయాలు కనుగొన్నవి: [ '/home/ravi/shared/ravi', '/home/asha/ravi/shared' ]
```

ఆరు దారులూ దొరికాయి – asha యొక్క file తో సహా, మరియు link ద్వారా వచ్చే ప్రత్యామ్నాయ దారులతో సహా. మరియు రెండు వలయాలూ పేరుతో సహా నివేదించబడ్డాయి.

ఇది ఒక్క FILE SYSTEM అలోచన కాదు - ఇది GRAPH TRAVERSAL యొక్క మౌలిక తేడా

DFS lo రెండు వేర్వేరు sets ఉంటాయి, మరియు అవి రెండు వేర్వేరు ప్రశ్నలకి జవాబిస్తాయి:

- **visited** ("ఎప్పుడైనా చూశామా") → **పనిని రెండుసార్లు చేయకుండా** ఆపుతుంది. ప్రతి node కి ఒక జవాబు ఉన్నప్పుడు సరైనది.
- **onPath** / recursion stack ("ఇప్పుడు ఈ దారిలో ఉందా") → **వలయాలని కనుగొంటుంది**. దారి కూడా జవాబులో భాగమైనప్పుడు సరైనది.

ఇదే తేడా **topological sort** lo (build dependencies, task scheduling), **deadlock detection** lo, **JSON circular reference** lo - అన్నిచోటా వస్తుంది. ఇక్కడ దారే జవాబు, కాబట్టి **onPath**.

మరియు నిజమైన systems ఇంకొక కవచం కూడా పెడతాయి: Linux lo ఒక path resolve చేసేటప్పుడు గరిష్ఠంగా **40 links** మాత్రమే అనుసరిస్తుంది - ఆ తర్వాత **EL00P**. §13 lo మనమూ ఒక హద్దు పెడతాం.

Part 5 – పూర్తి system

12. Step – ఒక subdirectory ని root గా చూపించడం

§8 lo `resolveInside` రాశాం. కానీ దాన్ని **ప్రతి call lo గుర్తుపెట్టుకుని వాడటం** ఒక సమస్య – ఎవరో ఒకచోట మర్చిపోతారు, మరియు అదే రంధ్రం.

మంచి రూపం: **ఆ పరిమితిని ఒక వస్తువుగా చేయడం.** `jail('/var/www')` ఒక చిన్న API ఇస్తుంది, అందులో బయటికి వెళ్ళే మార్గమే లేదు:

```
jail(rootPath) {
  const r = this.#lookup(rootPath, '/');
  if (r.err || !r.node.isDir) return null;
  const base = normalize('/', rootPath);
  const fence = base === '/' ? '/' : base + '/';
  return {
    resolve: (p) => {
      const full = normalize(base, p);
      if (full !== base && !full.startsWith(fence))
        return { ok: false, reason: `OUTSIDE_ROOT: ${full}` };
      return { ok: true, path: full };
    },
    read: (p) => {
      const rp = this.jail(rootPath).resolve(p);
      return rp.ok ? this.read(rp.path) : rp;
    },
  };
}
```

ఎందుకు ఒక **FLAG** కాదు, ఒక వస్తువు

`read(path, { jail: '/var/www' })` అని ఒక parameter పెట్టొచ్చు. కానీ అప్పుడు – **ఆ parameter ఇవ్వడం మర్చిపోవడం సాధ్యం**, మరియు మర్చిపోతే పూర్తి ప్రాప్యత వస్తుంది. **సురక్షితం కాని ప్రవర్తన default అవుతుంది.**

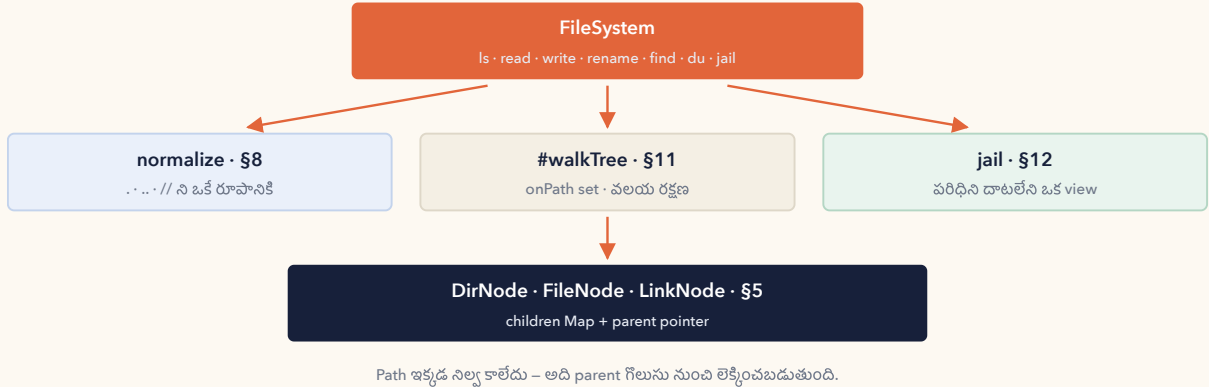
Jail ఒక విడి వస్తువు అయితే – server కి ఆ వస్తువుని మాత్రమే ఇస్తాం. దానికి పూర్తి `FileSystem` అసలు కనిపించదు. **మర్చిపోవడం సాధ్యం కాదు.**

ఇదే ఆలోచన Deep Dive 11 §10 lo చూసినది వేరే కోణం నుంచి: **తప్పుని పట్టుకోవడం కంటే, తప్పు చేయడమే వీలుకాకుండా చేయడం మేలు.**

13. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

నిర్మాణం · నాలుగు పొరలు

ప్రతి పొరా ఒక విరుపుకీ జవాబు



```
'use strict';
// ===== 1 · Nodes =====
class FsNode {
  constructor(name, parent = null) { this.name = name; this.parent = parent; }
  get isDir() { return false; }
  get isLink() { return false; }
  get path() {
    if (!this.parent) return '/';
    const up = this.parent.path;
    return up === '/' ? '/' + this.name : up + '/' + this.name;
  }
}
class FileNode extends FsNode {
  constructor(name, parent, content = '') {
    super(name, parent); this.content = content;
  }
  get size() { return this.content.length; }
}
class LinkNode extends FsNode {
  constructor(name, parent, target) { super(name, parent); this.target = target; }
  get isLink() { return true; }
  get size() { return 0; }
}
class DirNode extends FsNode {
  constructor(name, parent = null) {
    super(name, parent); this.children = new Map();
  }
  get isDir() { return true; }
  child(n) { return this.children.get(n) ?? null; }
  put(node) { this.children.set(node.name, node); return node; }
  drop(n) { return this.children.delete(n); }
}

// ===== 2 · Path =====
function normalize(base, path) {
  const raw = path.startsWith('/') ? path : base + '/' + path;
  const out = [];
  for (const p of raw.split('/')) {
    if (p === '' || p === '.') continue;
```

```

        if (p === '..') { out.pop(); continue; }
        out.push(p);
    }
    return '/' + out.join('/');
}

// ===== 3 · FileSystem =====
class FileSystem {
    constructor() { this.root = new DirNode(''); }

    #parts(path, cwd = '/') {
        return normalize(cwd, path).split('/').filter(Boolean);
    }

    #walk(parts, { create = false, followLinks = true } = {}) {
        let cur = this.root, hops = 0;
        for (const part of parts) {
            let next = cur.child(part);
            if (next && next.isLink && followLinks) {
                if (++hops > 32) return { err: 'TOO_MANY_LINKS' }; // వలయానికి కవచం
                next = next.target;
            }
            if (!next) {
                if (!create) return { err: `NOT_FOUND: ${part}` };
                next = cur.put(new DirNode(part, cur));
            }
            if (!next.isDir) return { err: `NOT_A_DIR: ${part}` };
            cur = next;
        }
        return { node: cur };
    }

    #lookup(path, cwd) {
        const parts = this.#parts(path, cwd);
        if (!parts.length) return { node: this.root };
        const name = parts.pop();
        const r = this.#walk(parts);
        if (r.err) return r;
        let node = r.node.child(name), hops = 0;
        while (node && node.isLink) {
            if (++hops > 32) return { err: 'TOO_MANY_LINKS' };
            node = node.target;
        }
        return node ? { node } : { err: `NOT_FOUND: ${path}` };
    }

    mkdirp(path, cwd = '/') {
        const r = this.#walk(this.#parts(path, cwd), { create: true });
        return r.err ? { ok: false, reason: r.err } : { ok: true, path: r.node.path };
    }

    write(path, content, cwd = '/') {
        const parts = this.#parts(path, cwd), name = parts.pop();
        const r = this.#walk(parts, { create: true });
        if (r.err) return { ok: false, reason: r.err };
        const existing = r.node.child(name);
        if (existing && existing.isDir) return { ok: false, reason: `IS_DIR: ${path}` };
        r.node.put(new FileNode(name, r.node, content));
        return { ok: true, path: `${r.node.path === '/' ? '' : r.node.path}/${name}` };
    }
}

```

```

read(path, cwd = '/') {
  const r = this.#lookup(path, cwd);
  if (r.err) return { ok: false, reason: r.err };
  if (r.node.isDir) return { ok: false, reason: `IS_DIR: ${path}` };
  return { ok: true, content: r.node.content };
}

ls(path = '/', cwd = '/') {
  const r = this.#lookup(path, cwd);
  if (r.err) return { ok: false, reason: r.err };
  if (!r.node.isDir) return { ok: true, entries: [r.node.name] };
  return { ok: true, entries: [...r.node.children.keys()].sort() };
}

link(from, to, cwd = '/') {                                     // from → to (symlink)
  const target = this.#lookup(to, cwd);
  if (target.err) return { ok: false, reason: target.err };
  const parts = this.#parts(from, cwd), name = parts.pop();
  const r = this.#walk(parts, { create: true });
  if (r.err) return { ok: false, reason: r.err };
  r.node.put(new LinkNode(name, r.node, target.node));
  return { ok: true, path: from };
}

rename(from, to, cwd = '/') {                                   // ఒక్క pointer మార్పు
  const fp = this.#parts(from, cwd), fname = fp.pop();
  const fr = this.#walk(fp);
  if (fr.err) return { ok: false, reason: fr.err };
  const node = fr.node.child(fname);
  if (!node) return { ok: false, reason: `NOT_FOUND: ${from}` };
  const tp = this.#parts(to, cwd), tname = tp.pop();
  const tr = this.#walk(tp, { create: true });
  if (tr.err) return { ok: false, reason: tr.err };
  if (node.isDir) {                                             // /a ని /a/b కిందకి జరపాల్సి
    for (let p = tr.node; p; p = p.parent)
      if (p === node) return { ok: false, reason: `LOOP: ${from} → ${to}` };
  }
  fr.node.drop(fname);
  node.name = tname; node.parent = tr.node;
  tr.node.put(node);
  return { ok: true, moved: node.path };
}

rm(path, cwd = '/') {
  const parts = this.#parts(path, cwd), name = parts.pop();
  const r = this.#walk(parts);
  if (r.err || !r.node.child(name))
    return { ok: false, reason: `NOT_FOUND: ${path}` };
  r.node.drop(name);
  return { ok: true };
}

// ---- వలయ-రక్షిత traversal ----
#walkTree(node, path, onPath, hit, loops) {
  if (node.isLink) {
    if (onPath.has(node.target)) { loops.push(path); return; }
    return this.#walkTree(node.target, path, onPath, hit, loops);
  }
  if (!node.isDir) { hit(node, path); return; }
  onPath.add(node);
  hit(node, path || '/');
  for (const [n, c] of node.children)
    this.#walkTree(c, `${path}/${n}`, onPath, hit, loops);
}

```

```

    onPath.delete(node);
  }
  find(path = '/', { match = () => true } = {}, cwd = '/') {
    const r = this.#lookup(path, cwd);
    if (r.err) return { ok: false, reason: r.err };
    const out = [], loops = [];
    const start = normalize(cwd, path) === '/' ? '' : normalize(cwd, path);
    this.#walkTree(r.node, start, new Set(),
      (n, p) => { if (!n.isDir && match(n, p)) out.push(p); }, loops);
    return { ok: true, files: out, loops };
  }
  du(path = '/', cwd = '/') {
    const r = this.#lookup(path, cwd);
    if (r.err) return { ok: false, reason: r.err };
    let bytes = 0;
    const seen = new Set();
    this.#walkTree(r.node, '', new Set(), (n) => {
      if (n.isDir || seen.has(n)) return; // ఒకే file రెండు దారుల్లో → ఒకసారే
      seen.add(n); bytes += n.size;
    }, []);
    return { ok: true, bytes };
  }
  jail(rootPath) {
    const r = this.#lookup(rootPath, '/');
    if (r.err || !r.node.isDir) return null;
    const base = normalize('/', rootPath);
    const fence = base === '/' ? '/' : base + '/';
    return {
      resolve: (p) => {
        const full = normalize(base, p);
        if (full !== base && !full.startsWith(fence))
          return { ok: false, reason: `OUTSIDE_ROOT: ${full}` };
        return { ok: true, path: full };
      },
      read: (p) => {
        const rp = this.jail(rootPath).resolve(p);
        return rp.ok ? this.read(rp.path) : rp;
      },
    };
  }
}

```

```

--- ls ---
/home          : [ 'asha', 'ravi' ]
/home/ravi     : [ 'empty', 'notes.txt', 'work' ] ← ఖాళీ directory కూడా
తప్పు దారి     : { ok: false, reason: 'NOT_FOUND: /home/nobody' }

--- paths ---
/home/ravi/notes.txt      → hello
/home//ravi//notes.txt   → hello
/home/./ravi/notes.txt   → hello
/home/asha/../ravi/notes.txt → hello

--- rename: ఒక్క pointer ---
{ ok: true, moved: '/home/asha/work' }
/home/ravi      : [ 'empty', 'notes.txt' ]
/home/asha     : [ 'resume.pdf', 'work' ]

```

```

report చదవడం : <pdf-bytes>
/a ని /a/b కిందకి: { ok: false, reason: 'LOOP: /home → /home/asha/home' }

--- links మరియు వలయం ---
/home/ravi/notes.txt
/home/ravi/shared/resume.pdf
/home/ravi/shared/work/report.pdf
/home/asha/resume.pdf
/home/asha/work/report.pdf
/home/asha/ravi/notes.txt
వలయాలు : [ '/home/ravi/shared/ravi', '/home/asha/ravi/shared' ]
du /home : 24 bytes (ఒకే file రెండుసార్లు కాదు)

--- jail: /var/www మాత్రమే ---
index.html { ok: true, content: '<h1>hi</h1>' }
./index.html { ok: true, content: '<h1>hi</h1>' }
.././etc/passwd { ok: false, reason: 'OUTSIDE_ROOT: /etc/passwd' }

```

ఈ output ని పంక్తి పంక్తిగా చదువుదాం

['empty', 'notes.txt', 'work'] – empty కనిపిస్తోంది. §4 lo అది ఉనికిలోనే లేదు.

నాలుగు paths, ఒకే జవాబు – //, ., .. అన్నీ ఒకే file కి చేరాయి.

LOOP: /home → /home/asha/home – /home ని తన సొంత మనవడి కిందకి జరపడానికి ప్రయత్నించాం.

అది జరిగితే ఆ subtree మొత్తం graph నుంచి తేగిపోయి ఒక వలయంగా మిగిలిపోతుంది. parent

గొలుసుని పైకి నడిచి ఆపేశాం – మూడు పంక్తులు.

du /home : 24 bytes – hello (5) + <pdf-bytes> (11) + <resume> (8) = 24. report.pdf

రెండు దారుల్లో కనిపించింది (/home/asha/work/ మరియు /home/ravi/shared/work/), కానీ ఒక్కసారే లెక్కించాం. find కి దారులు కావాలి; du కి files కావాలి – అందుకే find onPath వాడుతుంది, du దానికి తోడు seen కూడా వాడుతుంది.

OUTSIDE_ROOT: /etc/passwd – server కి /etc/passwd ఉందని కూడా తెలియదు.

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§3	Map<path, content>	మౌలిక అస్థిపంజరం
§4 (విరుపు)	DirNode · children Map · parent	ls 4,786 ms, rename 5 లక్షల keys
§5	path ఒక getter	Rename తర్వాత paths దానంతట అవే సర్వేనవి
§7 (విరుపు)	normalize	. .. పనిచేయవు; ఆపై /etc/passwd బయటికి
§8	fence తనిఖీ	సాధారణీకరణ సరిపోదు – పరిధి తనిఖీ కావాలి
§10 (విరుపు)	onPath set	Stack పేలింది; ఆపై ఒక file మాయమైంది
§11	hops > 32	Path resolution కి కూడా ఒక హద్దు

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§13	L00P తనిఖీ, du lo seen	Directory ని తన కిందకి జరపడం; రెండుసార్లు లెక్కింపు

Part 6 – Interview Io

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

45 నిమిషాల TIME BUDGET

మొదటి version వేగంగా రాయండి – నిజమైన సమయం తర్వాత మూడు ప్రశ్నలకే

5 నిమి

Clarify – ఖాళీ dir? rename? links? ..?

12 నిమి – Tree

Nodes · #walk · ls · rename · ఖర్చు గురించి మాట్లాడండి

10 నిమి – Paths

normalize + traversal దాడిని మరే లేవనెత్తండి

11 నిమి – Links

onPath vs visited · hop limit

7 నిమి

jail · du · permissions · persistence

ఏమి తప్పక చెప్పాలి

1. **Tree, Map కాదు** (§5) – మరియు ఎందుకో ఖర్చు పరంగా: "ls should cost the size of its answer." ఖాళీ directory ఉదాహరణ 15 సెకన్లు.
2. **Rename ఒక pointer మార్పు** (§5) – Map Io అది ప్రతి key. ఇది గుర్తుండిపోయే పోలిక.
3. **normalize ఆపై పరిధి తనిఖీ** (§8) – **path traversal ని మరే లేవనెత్తండి**, interviewer అడిగేదాకా ఆగొద్దు. ఇది ఒక పెద్ద సానుకూల సంకేతం.
4. **Symlinks అంటే ఇది graph** (§11) – onPath vs visited తేడా, మరియు visited వాడితే ఒక file నిశ్శబ్దంగా మాయమవుతుంది.
5. **Hop limit** (§13) – "Linux caps this at 40 and returns ELOOP."

ఏమి వదిలేయాలి

- **du, find, rm పూర్తిగా రాయొద్దు** – #walkTree ఒక్కటే రాసి, "find మరియు du ఇదే traversal, వేరే callback" అని చెప్పండి.
- **Hard links వదిలేయండి** – అడిగితే: "same inode, different names; delete drops a refcount."
- **Permissions** – ఒక వాక్యం (§16).
- **jail ని code గా రాయొద్దు** – ఆలోచన చెప్తే చాలు.

15. నీటితో చెప్పాల్సిన English script

"Four questions first. Can a directory be empty? Do you want rename or move? Should dot-dot work in paths? And are symlinks in scope?"

I ask about the empty directory because it decides everything. The obvious first design is a map from full path to content, and it passes the basic tests – but an empty directory has no content, so there's nothing to put in the map. It can't be represented at all.

The cost is the bigger problem. In a flat map, listing a directory means scanning every key in the whole file system. I measured it: listing a directory that always holds exactly twenty files took sixteen milliseconds at a thousand files and about forty-eight hundred at five hundred thousand – for the same twenty names. That breaks the rule I'd want to hold: an operation should cost the size of its answer, not the size of the database. And renaming one directory meant rewriting almost five hundred thousand keys, when in a real file system it's a single pointer change.

So I'd model it as a tree. A directory node holds a map from name to child, and every node holds a parent pointer. Listing is just the keys of one map. Rename unhooks the node from one parent and hooks it into another – the whole subtree comes along without moving, and paths stay correct because I compute path from the parent chain rather than storing it.

On paths – splitting on slash handles double slashes and trailing slashes for free, but dot and dot-dot need real handling, and that's where the security question lives. If you resolve dot-dot naively and stop there, a request for dot-dot-slash-dot-dot-slash-etc-slash-passwd resolves right out of your document root. That's path traversal, CWE-22. The fix isn't to reject paths containing dot-dot – encodings get around that. You normalise first, and then check that the normalised path is still inside the root, with a trailing slash on the prefix so that var-slash-www doesn't match var-slash-www-evil.

If symlinks are in scope, then this isn't a tree any more – it's a graph, and graphs have cycles. Two users linking to each other's folders is enough, and a recursive walk blows the stack. The tempting fix is a visited set, but that's wrong here: it stops the crash and then silently drops real files, because a directory reached once through a link is marked seen before you get to it by its own path. What you want is the set of directories on the current path – add on the way down, remove on the way back up. That reports the loops instead of hiding them, and it still finds every reachable file. It's the same distinction as cycle detection in topological sort. I'd also cap link hops the way Linux does, at forty, returning ELOOP.

One more thing I'd add: moving a directory into its own subtree has to be rejected, or you detach that subtree into a cycle. Walking up the parent chain of the destination is three lines."

16. Follow-ups – permissions, size cache, persistence

Follow-up	జవాబు	మారే classes
"Hard links"	<code>LinkNode</code> కాదు – అదే <code>node</code> ని రెండు <code>parents</code> lo పెట్టడం + <code>refcount</code>	<code>DirNode.drop</code>
" <code>find</code> with a pattern"	<code>match</code> callback ఇప్పటికే ఉంది	0
" <code>cd</code> మరియు relative paths"	<code>cwd</code> ఇప్పటికే ప్రతి method lo ఉంది	0
"Permissions (rwx)"	ప్రతి node కి <code>mode + owner</code> ; <code>#walk</code> lo ప్రతి directory కి <code>x</code> తనిఖీ	<code>FsNode</code> , <code>#walk</code>
"File watchers (<code>inotify</code>)"	Deep Dive 07 event bus – node మారినప్పుడు publish	0 concepts
"Directory size cache"	కింద చూడండి	<code>DirNode</code>
"Persistence / restart"	కింద చూడండి	కొత్త layer

Size cache – ఒక ఉచ్చుతో కూడిన follow-up

"ఇప్పుడు `du` ప్రతిసారీ మొత్తం subtree ని నడుస్తుంది. ప్రతి `DirNode` lo `cachedSize` ఉంచొచ్చు.

కానీ అప్పుడు ఒక file రాసినప్పుడు – దాని పై ఉన్న ప్రతి directory యొక్క cache చెల్లదు.

`parent` గొలుసు ఉంది కాబట్టి అది సులభం: పైకి నడిచి ప్రతిదాన్నీ `invalidate` చేయడం, $O(\text{లోతు})$.

మరియు ఇక్కడ symlinks మళ్ళీ కరుస్తాయి: ఒక file రెండు దారుల్లో ఉంటే, రెండు caches నీ `invalidate` చేయాలి – కానీ `parent` ఒక్కటే ఉంది. కాబట్టి **cache ని hard links / symlinks తో కలిపి వాడటం సురక్షితం కాదు**, లేదా ప్రతి node కి పలు parents ఉంచాలి.

ఇది చెప్పడం ముఖ్యం – cache ఒక సులభమైన జవాబులా కనిపిస్తుంది, కానీ **అది ఒక కొత్త స్థిరత్వ సమస్యని పుట్టిస్తుంది.**"

Persistence

"మొత్తం చెట్టుని `serialize` చేయొచ్చు, కానీ అది పెద్దదైతే ప్రతిసారీ ఖరీదు.

నిజమైన file systems రెండు ముక్కలుగా చేస్తాయి – **metadata** (ఏమి ఎక్కడ ఉంది: `inodes`, `directory entries`) మరియు **data blocks**. Metadata చిన్నది మరియు తరచుగా మారుతుంది; data పెద్దది.

మరియు స్థిరత్వం కోసం ఒక **journal** – "ఈ మార్పు చేయబోతున్నాను" అని ముందు రాసి, ఆపై చేయడం. మధ్యలో power పోతే, restart అయినప్పుడు journal చదివి పూర్తి చేయడం. ఇదే `ext4`, `NTFS`, మరియు ప్రతి database చేసేది."

17. ఏమి నేర్చుకున్నాం

ఆలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
ఖర్చు జనాబు పరిమాణంతో పెరగాలి	ls 4,786 ms → 1 ms (§5)	Deep Dive 08, 10 §8, 11 §8
నిర్మాణాన్ని నిల్వ చెయ్యి, తిరిగి కనుక్కోవద్దు	children + parent (§5)	Parsers, ASTs, org charts, comment threads
ఉత్పన్న విలువని నిల్వ చేయొద్దు	path ఒక getter (§5)	Deep Dive 05 (అప్పులు), 11 (bill)
సాధారణీకరించు → అపై ధ్రువీకరించు	normalize + fence (§8)	URLs, emails, Unicode, SQL
visited vs onPath	ఒక file మాయమైంది (§11)	Topological sort, deadlock detection
ప్రతి recursion కి ఒక హద్దు	hops > 32 (§13)	Redirects, retries, JSON depth
సురక్షితం కానిది default కాకూడదు	jail ఒక వస్తువు (§12)	API design, feature flags

ఒక చివరి ఆలోచన

ఈ problem ని చాలా మంది "సులభమైనది" అంటారు, మరియు LeetCode మీద అది నిజమే.

కానీ ఇక్కడ చూసినవి గమనించండి: ఒక ఖర్చు సమస్య (§4), ఒక భద్రతా దుర్బలత్వం (§7), ఒక graph వలయం (§10), ఒక నిశ్శబ్ద డేటా నష్టం (§10), మరియు ఒక cache స్థిరత్వ ఉచ్చు (§16).

"సులభమైన" problems lo interviewer చూసేది పరిష్కారం కాదు – మీరు ఎంత లోతుకి వెళ్తారో. ఒక Map రాసి ఆగిపోతే అది ఒక జనాబు. ఖర్చుని కొలిచి, దాడిని మీరే లేవనెత్తి, వలయాన్ని ముందే చూస్తే – అది వేరే స్థాయి.

ఇక్కడి నుంచి ఎక్కడికి

ఈ series lo ఇప్పటివరకు: **01 Parking Lot · 02 Cache · 03 Rate Limiter · 04 BookMyShow · 05 Splitwise · 06 Elevator · 07 Pub-Sub · 08 HashMap · 09 Chess · 10 Meeting Scheduler · 11 Food Delivery · 12 File System.**

వేగవంతమైన revision కోసం – [LLD_Design_Problems_Telugu.pdf](#) .

In-Memory File System – అడుగు అడుగునా · ఈ doc lo ఉన్న ప్రతి output, ప్రతి సమయం, ఆ RangeError తో సహా నిజంగా node lo run చేసి తీసినదే ✓